

Implementation Roadmap — Recommendation 1 (Sprint / Feature / Task Domain + API)

Source of truth: [docs/migration-recommendation-1.md](#) — **finalized, do not redesign**. Status: **Roadmap — no code written yet.**

This roadmap converts the approved migration plan into small, independently buildable / testable / reviewable / mergeable milestones. Each task is executed one at a time under the Execution Rules (§2). Any deviation from the approved plan is only allowed if a critical implementation issue is discovered; it must then be surfaced before proceeding.

1. Preconditions (must be confirmed before Phase 1)

Decisions (a)–(e) from the migration plan §2 must be signed off:

Decision	Recommended	Blocks
(a) Single Task collection	A — extend existing Task	All
(b) Feature.sprintRef nullable (backlog)	yes	Phase 1
(c) Delete = null-out refs, no cascade	yes	Phase 3
(d) Extend Project model now (Phase 3.5)	defer unless project cards need it	optional task P1-T4
(e) Adopt §9 additions (Backlog, types, ownership invariants)	yes	Phase 1, 3, 6

Environment notes (verified):

- Server: Node/Express/Mongoose; frontend: React + Zustand + Vite.
- `npm run typecheck` + `npm test` (frontend) run from `mainApp`; server tests: `npx vitest run --root . server/__tests__` from repo root.
- Migrations are forward-only and idempotent (`server/migrations/core.js`); apply via `node migrations/run.js --apply --db=<URI>` (dry-run first).
- No `rg` binary — use `grep/READ` tools.

2. Execution Rules (apply to every task)

1. Implement **exactly one task**.
2. Build the project (typecheck / server boot).

3. Run the full relevant test suites (frontend `npm test` , server vitest).
4. Verify functionality manually per the task's Verification steps.
5. Summarize the change (files, behavior, test results).
6. Recommend the next task.
7. **STOP** — never continue automatically.

Per-task loop template:

```
[TASK-ID] <title>
1. implement → 2. npm run typecheck → 3. npm test (+ npx vitest run --root .
server/__tests__)
4. manual verification per steps → 5. summary → 6. next task → 7. stop
```

Guardrails for every change:

- Extend existing models; never create duplicate systems.
- Keep personal-task behavior byte-for-byte identical (new fields default to null/empty).
- All migrations idempotent; single release; DB backup before `--apply` .
- Hierarchy invariant: `workspace` → `Project` → `(Backlog)` → `Sprint` → `Feature` → `Task` → `Session` → `Work Log` ; `workspaceRef` is always server-derived from the owning `Project`.

3. Phase Map

Phase	Title	Depends on	Effort
1	Database models	—	M
2	Database migrations	1	S
3	Backend APIs	1 (2 optional)	L
4	Frontend API integration (types + client)	3 (contract)	M
5	Collaboration store (API-backed)	4	M
6	UI integration	5	L
7	Testing	3, 5	M
8	Verification & backward-compat sign-off	all	S–M

4. Phase Details

Phase 1 — Database models

- **Objective:** Add `Sprint` and `Feature` models; extend `Task` with collaboration refs/fields and indexes. No runtime behavior changes.
- **Scope:** New models + model extension only. Frontend untouched.
- **Files to modify:**
 - `server/models/Sprint.js` (new)
 - `server/models/Feature.js` (new)
 - `server/models/Task.js` (extend)
 - `server/models/Project.js` (optional, only if decision (d) = now)
- **Dependencies:** None. Precondition decisions (a), (b), (e).
- **Risks:** Adding fields to `Task` could interact with existing route tests — new fields must all have defaults so `Task.create` with legacy payloads is unchanged. Mongoose auto-index at boot is idempotent alongside migration 0010.
- **Acceptance criteria:** Models import cleanly; `Task` schema still accepts legacy create payloads; existing server test suite green.
- **Definition of Done:** `Sprint`, `Feature` fields and `Task` additions match migration plan §3.1–3.3 exactly; indexes declared per plan.
- **Estimated effort:** M (~4–6h).
- **Testing strategy:** No new tests; regression = existing `server/__tests__` suite (398 tests / 38 files).
- **Rollback:** Pure code addition; revert the three file changes with zero data impact.

Tasks

P1-T1 — Create `Sprint` model

- **Description:** Implement `server/models/Sprint.js` per plan §3.1 (`projectRef` required, `workspaceRef` required, `name`, `goal`, `startDate`, `endDate`, `status` enum `['future', 'active', 'completed']`, `capacityHours`, `targetVelocity`, `createdBy`, `timestamps`) with indexes `{projectRef:1, startDate:-1}`, `{workspaceRef:1, status:1}`.
- **Affected files:** `server/models/Sprint.js` (new).
- **Database impact:** New `sprints` collection created on first model use / migration; no existing data touched.
- **API impact:** None yet.
- **Frontend impact:** None.
- **Testing required:** Existing server suite green; manual import smoke test.
- **Verification steps:** Boot server or run a Node one-liner requiring the model; confirm schema paths and indexes.

- Suggested commit message: `feat(server): add Sprint model (R1-P1)`

P1-T2 — Create Feature model

- Description: Implement `server/models/Feature.js` per plan §3.2 including backlog/work-item additions (`sprintRef` default null, type enum with default `feature`, `labels`, `ownerId`, `estimatedHours`, `status`, `order`) and indexes `{projectRef:1, order:1}`, `{sprintRef:1, status:1}`, `{workspaceRef:1}`, `{type:1}`.
- Affected files: `server/models/Feature.js` (new).
- Database impact: New `features` collection; no existing data touched.
- API impact: None yet.
- Frontend impact: None.
- Testing required: Existing suite green; import smoke test.
- Verification steps: Require model; verify enum defaults and index declarations.
- Suggested commit message: `feat(server): add Feature model with backlog + work-item fields (R1-P1)`

P1-T3 — Extend Task model for collaboration

- Description: Append to `server/models/Task.js` per plan §3.3: `workspaceRef`, `projectRef`, `sprintRef`, `featureRef` (all refs, default null), `assigneeId`, `reviewerId`, `followerIds`, `labels`, `dependencies`, `estimatedHours`, `actualHours`, `sprintStatus` enum, `gitContext` subdoc; add indexes `{workspaceRef:1, sprintRef:1}`, `{featureRef:1}`, `{projectRef:1}`. All defaults must preserve legacy behavior.
- Affected files: `server/models/Task.js`.
- Database impact: No existing docs modified; new fields default to null/empty; indexes created on next sync (or migration 0011).
- API impact: None yet (routes untouched).
- Frontend impact: None.
- Testing required: Full existing server suite — especially `tasks / sessions / worklogs` route tests that create tasks with legacy payloads.
- Verification steps: `npx vitest run --root . server/__tests__`; confirm all task-related tests pass unchanged.
- Suggested commit message: `feat(server): extend Task with collaboration refs (R1-P1)`

P1-T4 (OPTIONAL — only if decision (d) = now) — Extend Project model

- Description: Persist `description`, `key`, `status`, `milestones`, `members`, `teamIds` per plan §3.4.
- Affected files: `server/models/Project.js`.
- Database impact: New optional fields; no backfill (defaults).
- API impact: `POST/PATCH /api/projects` schema extension (defer to Phase 3).

- Frontend impact: `toProject` mapper already tolerates missing fields (`useCollaborationStore.ts:77-91`).
- Testing required: Existing project route tests green.
- Verification steps: Create a project via API; confirm new fields persist.
- Suggested commit message: `feat(server): persist project metadata fields (R1-P1-opt)`

Phase 2 — Database migrations

- **Objective:** Versioned, idempotent creation of `sprints` / `features` indexes and safe backfill of legacy tasks with collab defaults.
- **Scope:** Two migration files only.
- **Files to modify:**
 - `server/migrations/migrations/0010_create_sprint_feature_collections.js` (new)
 - `server/migrations/migrations/0011_task_collab_links.js` (new)
- **Dependencies:** Phase 1 (schema names must match).
- **Risks:** 0011 `updateMany({}, $set ...)` touches all tasks — must be guarded (skip docs already migrated / idempotent by design). Backup DB before `--apply`.
- **Acceptance criteria:** `--dry-run` lists exactly the two pending files; `--apply` succeeds; re-running reports no pending migrations.
- **Definition of Done:** Collections/indexes exist in a target DB; legacy tasks have `workspaceRef: null` + collab defaults; no existing task data lost.
- **Estimated effort:** S (~2–3h).
- **Testing strategy:** Apply against a disposable local DB twice (idempotency), plus the existing migration-framework unit tests if present.
- **Rollback:** Forward-only (`core.js` has no `down`). Rollback = manual `dropIndex / $unset` + code revert. Plan single release.

Tasks

P2-T1 — Migration 0010: create sprint/feature collections + indexes

- Description: `0010_create_sprint_feature_collections.js` — `createIndex` on `sprints` and `features` (auto-creates collections) per plan §4; guard for idempotency; return a summary count.
- Affected files: `server/migrations/migrations/0010_create_sprint_feature_collections.js` (new).
- Database impact: Creates `sprints` + `features` collections and indexes; non-destructive.
- API impact: None.
- Frontend impact: None.
- Testing required: `node migrations/run.js --dry-run`; apply on a scratch DB; run twice.

- Verification steps: Confirm collections + indexes via `mongo shell / driver`; `--dry-run` shows `No pending migrations` after apply.
- Suggested commit message: `feat(db): add sprint/feature collections migration (R1-P2)`

P2-T2 — Migration 0011: backfill task collaboration links

- Description: `0011_task_collab_links.js` — `tasks.updateMany({}, { $set: { workspaceRef:null, projectRef:null, sprintRef:null, featureRef:null, sprintStatus:'backlog', labels:[], dependencies:[], followerIds:[], estimatedHours:0, actualHours:0 } })` per plan §4.
- Affected files: `server/migrations/migrations/0011_task_collab_links.js` (new).
- Database impact: Writes collab defaults to every existing task; no data loss.
- API impact: None.
- Frontend impact: None.
- Testing required: Idempotency check (run twice, zero extra writes); sample-count before/after.
- Verification steps: Confirm a legacy task now has `workspaceRef: null` and `sprintStatus: 'backlog'`.
- Suggested commit message: `feat(db): backfill task collab defaults migration (R1-P2)`

Phase 3 — Backend APIs

- **Objective:** Extract shared workspace gates; add `sprints` + `features` routes; extend `tasks.js` for workspace-scoped collaboration; enforce ownership invariants.
- **Scope:** Server routes + middleware only. No frontend changes yet.
- **Files to modify:**
 - `server/middleware/workspace.js` (add gates)
 - `server/routes/sprints.js` (new)
 - `server/routes/features.js` (new)
 - `server/index.js` (mount routes)
 - `server/routes/tasks.js` (extend)
- **Dependencies:** Phase 1 (models); Phase 2 recommended (so all legacy tasks carry `workspaceRef: null` before the personal GET query changes).
- **Risks:** Highest-risk phase — `tasks.js` backward compatibility. Personal paths must remain identical; `GET / personal` query changes from `{userId}` to `{userId, workspaceRef:null}` — safe only after migration 0011 (all tasks have `workspaceRef:null`). Ownership invariants must be enforced on every new route.
- **Acceptance criteria:** All new endpoints respond correctly with proper gates; legacy personal task CRUD unchanged; existing task/session route tests green.
- **Definition of Done:** Sprint/Feature CRUD + backlog query + task workspace filters live; `workspaceRef` never accepted from client body; Activity writes on create; refs share a single

`projectRef` .

- **Estimated effort:** L (~10–14h).
- **Testing strategy:** New supertest route tests written in this phase (or Phase 7); regression suite green each task.
- **Rollback:** Code revert is clean (no data writes except Activity rows, which are append-only and TTL'd).

Tasks

P3-T1 — Extract shared workspace gates

- Description: Add to `server/middleware/workspace.js` reusable gates for the new flat routes: `requireWorkspaceMember` , `requireWorkspaceEditor` (non-Viewer), `requireWorkspaceOwnerAdmin` (Owner/Admin), backed by one `Workspace.findById().select('members') + findMember` , plus a `resolveProjectWorkspace` helper (loads Project by id, sets `req.project` + `req.workspace` from `project.workspaceRef`). Do **not** refactor existing `projects.js` / `teams.js` gates in this task (protect current behavior).
- Affected files: `server/middleware/workspace.js` .
- Database impact: None.
- API impact: New middleware/helpers available; no endpoint behavior change.
- Frontend impact: None.
- Testing required: New unit/integration tests for each gate (member, non-Viewer editor, Owner/Admin).
- Verification steps: Write throwaway route or reuse Phase 3 tests; confirm 403/200 matrix.
- Suggested commit message: `feat(server): extract reusable workspace gates (R1-P3)`

P3-T2 — Sprint routes

- Description: `server/routes/sprints.js` per plan §5 — `GET /?projectId=` (member), `POST /` (editor; validate `startDate < endDate` ; derive `workspaceRef` from project; write `Activity('sprint.created')`), `PATCH /:id` (editor; scoped by `workspaceRef` ; goal/dates/capacity/targetVelocity/status), `DELETE /:id` (Owner/Admin; nulls `sprintRef` on features and tasks via `updateMany`).
- Affected files: `server/routes/sprints.js` (new).
- Database impact: Inserts/updates `sprints` ; `DELETE` nulls `sprintRef` on `features` / `tasks` (no cascade).
- API impact: New `/api/sprints` surface.
- Frontend impact: None (client added Phase 4).
- Testing required: Route tests — gates, date validation, workspace scoping, delete null-out.
- Verification steps: Manual curl/API calls against a dev server with a seeded workspace.
- Suggested commit message: `feat(server): add sprint CRUD routes (R1-P3)`

P3-T3 — Feature routes (incl. backlog)

- Description: `server/routes/features.js` per plan §5 — `GET /?projectId=&sprintId=&backlog=true&type=&status=` (member; `backlog=true` ⇒ `sprintRef:null`; validate sprint belongs to project), `POST /` (editor), `PATCH /:id` (editor; `name/description/type/labels/ownerId/estimatedHours/status/order/ sprintRef`; `sprintRef` change revalidates same-project), `DELETE /:id` (Owner/Admin; nulls `featureRef` on tasks).
- Affected files: `server/routes/features.js` (new).
- Database impact: Inserts/updates `features`; DELETE nulls `featureRef` on `tasks`.
- API impact: New `/api/features` surface; backlog query via `sprintRef:null`.
- Frontend impact: None yet.
- Testing required: Backlog query, `sprintRef` validation, filters, gates.
- Verification steps: Create features, move into/out of sprint via PATCH, confirm backlog query.
- Suggested commit message: `feat(server): add feature CRUD + backlog routes (R1-P3)`

P3-T4 — Mount new routes

- Description: Add `app.use('/api/sprints', sprintRoutes)` and `app.use('/api/features', featureRoutes)` to `server/index.js` after line 125 (per plan §5).
- Affected files: `server/index.js`.
- Database impact: None.
- API impact: Routes live behind `/api`.
- Frontend impact: None.
- Testing required: Server boot + smoke calls.
- Verification steps: `GET /api/sprints` and `/api/features` return structured responses (401 without auth).
- Suggested commit message: `feat(server): mount sprint/feature routes (R1-P3)`

P3-T5 — Extend tasks routes (workspace scoping + git)

- Description: Extend `server/routes/tasks.js` per plan §5 — `GET /` adds `?workspaceId= / ?projectId= / ?sprintId= / ?featureId=` filters (member-gated when workspace-scoped; personal path becomes `{userId, workspaceRef:null}`); `POST /` accepts optional refs (derive `workspaceRef`, validate refs share one `projectRef`, editor gate); `PATCH / DELETE` keep `{_id,userId}` scope and add editor/OwnerAdmin gate when task is workspace-scoped; new `PATCH /:id/git` for `gitContext`.
- Affected files: `server/routes/tasks.js`.
- Database impact: Writes collab fields on task create/update.
- API impact: Backward-compatible task API + workspace filters + git endpoint.
- Frontend impact: None yet.
- Testing required: Regression on all existing task route tests; new tests for filters, gates, git patch, cross-project rejection.

- Verification steps: Personal task flow unchanged; workspace task CRUD works with member gates.
- Suggested commit message: `feat(server): workspace-scoped tasks + git context (R1-P3)`

Phase 4 — Frontend API integration (types + client)

- **Objective:** Define `Feature / FeatureType` types, align `CollaborativeTask / Sprint`, and add the API client surfaces for sprints/features/tasks.
- **Scope:** Types + `api.ts` only. No store/UI behavior changes.
- **Files to modify:**
 - `src/types/collaboration.ts`
 - `src/utils/api.ts`
- **Dependencies:** Phase 3 (API contract), decision (e).
- **Risks:** Changing `CollaborativeTask` field semantics (`refs` → `ids`) may break existing pages that read the old shape — keep fields compatible during transition or update consumers in Phase 6. Use additive fields where possible.
- **Acceptance criteria:** Typecheck passes; `api` object exposes `sprints / features` and extended `tasks`.
- **Definition of Done:** Frontend can call every new/updated endpoint with correct request/response shapes.
- **Estimated effort:** M (~4–6h).
- **Testing strategy:** `npm run typecheck`; `api.collaboration.test.ts` extended with mocked request layer.
- **Rollback:** Code revert; no data impact.

Tasks

P4-T1 — Add collaboration types

- Description: Add `Feature` interface (`id`, `projectId`, `sprintId?`, `workspaceId`, `name`, `description`, `type`, `labels`, `ownerId?`, `estimatedHours`, `status`, `order`, `createdAt`), `FeatureType` union (`'feature'` | `'bug'` | `'spike'` | `'chore'` | `'research'` | `'debt'` | `'improvement'`), and align `CollaborativeTask` to the server refs (`workspaceId`, `projectId`, `sprintId?`, `featureId?`). Per plan §6.
- Affected files: `src/types/collaboration.ts`.
- Database impact: None.
- API impact: None.
- Frontend impact: Type surface only; adjust consumers in Phase 6 to avoid type errors.
- Testing required: `npm run typecheck`.
- Verification steps: Typecheck clean; update any consumer import errors surfaced (note them for Phase 6).

- Suggested commit message: `feat(web): add Feature/FeatureType collaboration types (R1-P4)`

P4-T2 — API client surfaces

- Description: In `src/utils/api.ts` add `sprints: {list,create,update,remove}`, `features: {list,create,update,remove}` (pattern from `api.ts:301-309`) and extend `tasks` with workspace filters, collab create/update payloads, and `patchGit`. Per plan §6.
- Affected files: `src/utils/api.ts`.
- Database impact: None.
- API impact: Frontend now targets the Phase 3 endpoints.
- Frontend impact: New client methods available.
- Testing required: Extend `api.collaboration.test.ts` to mock the request layer and assert URL/method/body.
- Verification steps: Typecheck + client tests green.
- Suggested commit message: `feat(web): add sprint/feature API client (R1-P4)`

Phase 5 — Collaboration store (API-backed)

- **Objective:** Replace mock sprint/task state with API loaders and optimistic `runMutation` actions; remove hardcoded `'m1'`; delete dead mock bodies.
- **Scope:** `useCollaborationStore.ts` only.
- **Files to modify:**
 - `src/store/useCollaborationStore.ts`
- **Dependencies:** Phase 4 (client + types).
- **Risks:** Store is consumed by many pages (TeamWorkspace, FeaturesPage, QA, Reports, modals). Swapping actions to async API calls changes call sites' return types (`createTask` returns a Promise now vs a Task synchronously) — update consumers in Phase 6; keep store contract compatible (return the created entity).
- **Acceptance criteria:** Store loaders populate sprints/features/tasks from the API; create/update actions persist and roll back on failure; no `'m1'` literals remain.
- **Definition of Done:** `loadCollabData` includes the three new loaders; sprint/feature/task actions are `runMutation`-based; mock-only bodies removed.
- **Estimated effort:** M (~6–8h).
- **Testing strategy:** `useCollaborationStore.test.ts` extended (mock `api`, assert loaders + optimistic rollback).
- **Rollback:** Code revert; store state is transient.

Tasks

P5-T1 — Mappers + state fields

- Description: Add `toSprint`, `toFeature`, `toCollabTask` mappers (mirror `toProject`, `useCollaborationStore.ts:77-91`); ensure `sprints`, `tasks`, `features` state shapes match Phase 4 types.
- Affected files: `src/store/useCollaborationStore.ts`.
- Database impact: None.
- API impact: None.
- Frontend impact: Normalized store shapes for new entities.
- Testing required: Typecheck; mapper unit tests.
- Verification steps: Typecheck + store tests green.
- Suggested commit message: `feat(web): add collab store mappers (R1-P5)`

P5-T2 — Loaders

- Description: Add `loadSprints` / `loadFeatures` / `loadTasks` wired into `loadCollabData` (`useCollaborationStore.ts:243-250`). Loads are keyed off `activeWorkspaceId`; empty/offline-safe (mirror `loadProjects`).
- Affected files: `src/store/useCollaborationStore.ts`.
- Database impact: None.
- API impact: Consumes Phase 4 client.
- Frontend impact: Pages receive real data after workspace load.
- Testing required: Store tests mock `api.sprints/features/tasks` and assert state population + empty fallback.
- Verification steps: Navigate to a workspace; devtools show populated sprints/tasks/features.
- Suggested commit message: `feat(web): collab store loaders for sprints/features/tasks (R1-P5)`

P5-T3 — API-backed actions

- Description: Convert `createSprint` (:408), `createTask` (:426), `updateTaskStatus` (:454), `updateGitContext` (:466) to `runMutation` optimistic actions that persist via the Phase 4 client and roll back on failure. Keep return shapes compatible with current call sites.
- Affected files: `src/store/useCollaborationStore.ts`.
- Database impact: None.
- API impact: Writes go through new endpoints.
- Frontend impact: Sprint/board/task interactions now persist.
- Testing required: Store tests — optimistic set, API call shape, rollback on error.
- Verification steps: Create a sprint/task; refresh; data persists.
- Suggested commit message: `feat(web): API-back collab sprint/task actions (R1-P5)`

P5-T4 — Real user identity + remove mocks

- **Description:** Replace hardcoded `'m1'` (`createTask :437-440`, `comments`, `docs`, `blockers`) with the authenticated user (via `auth/ api.me`) and real `members` for assignee/reviewer; delete dead mock-only helper bodies and seed remnants (`comments useCollaborationStore.ts:116, 143`).
- **Affected files:** `src/store/useCollaborationStore.ts`.
- **Database impact:** None.
- **API impact:** Payloads carry real user/member ids.
- **Frontend impact:** Ownership/across-member data becomes truthful.
- **Testing required:** Grep assert no `'m1'` remains; store tests updated.
- **Verification steps:** Create a task; confirm `ownerId` = logged-in user id.
- **Suggested commit message:** `refactor(web): remove hardcoded member mocks (R1-P5)`

Phase 6 — UI integration

- **Objective:** All workspace pages consume live store data; add Backlog view + type badges; fix modals.
- **Scope:** Workspace pages/components only. No new routes (drill-down navigation is Rec 4, out of scope).
- **Files to modify:**
 - `src/pages/collaboration/TeamWorkspace.tsx`
 - `src/pages/collaboration/FeaturesPage.tsx`
 - `src/pages/collaboration/ReportsAnalyticsPage.tsx`
 - Backlog view component (new, e.g. `src/components/collaboration/ProjectBacklog.tsx`)
 - Type badge/filter component (new, e.g. `src/components/collaboration/WorkItemTypeBadge.tsx`)
 - Create-Sprint / Create-Task modals (`src/components/collaboration/*Modal*`)
- **Dependencies:** Phase 5.
- **Risks:** Board/kanban logic (`TeamWorkspace.tsx:336-355, 400-414`), velocity/capacity cards (`:235-236, 346-352`), `FeaturesPage` (`:25-31, 72, 141-171`), and `ReportsAnalyticsPage` (`:15-19, 30-33`) all change data source from mock to live — must handle empty/loading states so pages don't crash offline.
- **Acceptance criteria:** Every workspace page renders live data; no mock data visible; empty states are graceful.
- **Definition of Done:** Backlog view (ordered, filterable, drag-drop into sprint), type badges/filters, real assignee dropdowns, live velocity/completion-rate (placeholders only where Rec 6 is explicitly deferred).
- **Estimated effort:** L (~10–14h).
- **Testing strategy:** Component tests where feasible; manual workspace walkthrough; `a11y. axe` suite still green.
- **Rollback:** Code revert; no data impact.

Tasks

P6-T1 — TeamWorkspace sprint board + cards consume live data

- Description: Sprint board columns (`TeamWorkspace.tsx:336-355`), task cards (`:400-414`), velocity/capacity cards (`:235-236, 346-352`) read from real store `sprints / tasks` . Add empty/loading states.
- Affected files: `src/pages/collaboration/TeamWorkspace.tsx` .
- Database impact: None.
- API impact: None.
- Frontend impact: Board reflects persisted data; status changes persist via Phase 5 actions.
- Testing required: Component smoke tests; manual board flow.
- Verification steps: Create task → appears on board → refresh persists.
- Suggested commit message: `feat(web): wire sprint board to live data (R1-P6)`

P6-T2 — FeaturesPage live features

- Description: Render real `features` (`FeaturesPage.tsx:72`); "Private Implementation Tasks" lists tasks by `featureRef` (`:141-171`); type badges/filters (new `WorkItemTypeBadge`).
- Affected files: `src/pages/collaboration/FeaturesPage.tsx` ; new `src/components/collaboration/WorkItemTypeBadge.tsx` .
- Database impact: None.
- API impact: None.
- Frontend impact: Features page truthful; tasks link to feature via `featureRef` .
- Testing required: Component tests + manual.
- Verification steps: Create feature with type → badge shows; tasks grouped under feature.
- Suggested commit message: `feat(web): wire Features page + work-item type badges (R1-P6)`

P6-T3 — Project Backlog view

- Description: New `ProjectBacklog` component: backlog features (`sprintRef == null`) ordered by `order` ; filter by `status / type / owner / labels` ; client-side search; drag-and-drop into a Sprint (persists via `PATCH features/:id {sprintRef}`). Placed within the workspace features/board area (URL-tethered navigation deferred — Rec 4). Per plan §9.1/§6.
- Affected files: `src/components/collaboration/ProjectBacklog.tsx` (new); integration point in `TeamWorkspace.tsx` or `FeaturesPage.tsx` .
- Database impact: None.
- API impact: Uses `GET /features?backlog=true` + `PATCH /features/:id` .
- Frontend impact: First-class per-Project backlog.
- Testing required: Backlog query mock; drag-drop handler tests; filter tests.

- Verification steps: Backlog shows sprint-less features; dropping a feature into a sprint removes it from backlog and persists.
- Suggested commit message: `feat(web): add Project Backlog view (R1-P6)`

P6-T4 — Modals with real members

- Description: Create-Sprint / Create-Task modals use real `members` for assignee/reviewer (no `'m1'`); feature-aware task creation (assign `featureRef`).
- Affected files: `src/components/collaboration/*Modal*` (CreateSprint, CreateTask).
- Database impact: None.
- API impact: None.
- Frontend impact: Correct ownership from creation.
- Testing required: Modal form tests.
- Verification steps: Assign a real member; task persists with their id.
- Suggested commit message: `feat(web): wire sprint/task modals to real members (R1-P6)`

P6-T5 — ReportsAnalyticsPage live KPIs

- Description: Feature completion rate (`ReportsAnalyticsPage.tsx:15-19`) and velocity (`:30-33`) from live feature/task data; keep explicit placeholders only for metrics deferred to Rec 6 (cycle time, burndown).
- Affected files: `src/pages/collaboration/ReportsAnalyticsPage.tsx`.
- Database impact: None.
- API impact: None.
- Frontend impact: KPI numbers become real.
- Testing required: Unit tests for the two computations.
- Verification steps: Complete a feature → completion rate reflects it.
- Suggested commit message: `feat(web): live workspace KPIs in ReportsAnalytics (R1-P6)`

Phase 7 — Testing

- **Objective:** Comprehensive automated coverage for the new domain + regression proof for the personal stack.
- **Scope:** New server route tests, store/client test updates, backlog behavior tests.
- **Files to modify:**
 - `server/__tests__/sprints.test.js` (new)
 - `server/__tests__/features.test.js` (new)
 - `server/__tests__/workspaceTasks.test.js` (new)
 - `src/store/__tests__/useCollaborationStore.test.ts` (extend)
 - `src/utils/__tests__/api.collaboration.test.ts` (extend)
 - `src/utils/__tests__/api.reports.test.ts` (only if touched)

- **Dependencies:** Phases 3 and 5.
- **Risks:** Tests must not depend on ordering; use in-memory DB / mocks consistent with existing conventions.
- **Acceptance criteria:** New coverage for: gates (Viewer read vs Editor write), same-project validation, workspace scoping, backlog query, delete null-out, personal-task backward compat, store loaders + optimistic rollback.
- **Definition of Done:** `npx vitest run --root . server/__tests__` and frontend `npm test` both green with new tests included.
- **Estimated effort:** M (~6–8h).
- **Testing strategy:** Mirror existing route/store test patterns (see `server/__tests__` and `useCollaborationStore.test.ts:4-23, 62-68`).
- **Rollback:** Tests only; revert freely.

Tasks

P7-T1 — Server route tests (sprints/features/workspaceTasks)

- Description: Supertest suites covering the §5 matrix: membership gates, editor vs Viewer, Owner/Admin delete, `startDate < endDate`, feature ↔ sprint same-project, backlog query, task workspace filters, cross-project rejection, and personal-path regression.
- Affected files: `server/__tests__/sprints.test.js`, `features.test.js`, `workspaceTasks.test.js` (new).
- Database impact: None.
- API impact: None.
- Frontend impact: None.
- Testing required: Full new suites + existing regression.
- Verification steps: `npx vitest run --root . server/__tests__`.
- Suggested commit message: `test(server): sprint/feature/workspace-task routes (R1-P7)`

P7-T2 — Store + client tests

- Description: Extend `useCollaborationStore.test.ts` (loaders populate; optimistic rollback on error; no `'m1'`) and `api.collaboration.test.ts` (new client method shapes).
- Affected files: `src/store/__tests__/useCollaborationStore.test.ts`; `src/utils/__tests__/api.collaboration.test.ts`.
- Database impact: None.
- API impact: None.
- Frontend impact: None.
- Testing required: Store + API client suites.
- Verification steps: Frontend `npm test` green.

- Suggested commit message: `test(web): collab store loaders/actions + api client (R1-P7)`

Phase 8 — Verification & sign-off

- **Objective:** Full-project verification and explicit proof the personal stack is unaffected.
- **Scope:** No new features; validation only.
- **Files to modify:** None (documentation/notes only).
- **Dependencies:** All phases.
- **Risks:** Integration edge cases surfaced only at this stage (offline empty states, index build timing in prod).
- **Acceptance criteria:** Every success criterion below demonstrably met.
- **Definition of Done:** Signed-off checklist + runbook note for single-release migration.
- **Estimated effort:** S–M (~3–5h).
- **Testing strategy:** Full suites + manual scripted walkthrough.
- **Rollback:** N/A (validation).

Tasks

P8-T1 — Full verification pass

- Description: Run `npm run typecheck`, frontend `npm test`, `npx vitest run --root . server/__tests__`, migration `--dry-run` then `--apply` on a staging DB; scripted manual walkthrough: workspace → project → backlog → sprint → feature → task → (Rec 2 link when available) → session → worklog.
- Affected files: none.
- Database impact: Staging DB only.
- API impact: None.
- Frontend impact: None.
- Testing required: All suites + manual script.
- Verification steps: See DoD checklist below.
- Suggested commit message: `chore: R1 verification pass`

P8-T2 — Personal-stack regression sign-off

- Description: Prove personal-task functionality is byte-for-byte unaffected: create/complete tasks, run sessions, sync worklogs, day/week totals, reports, share — all against the same code that now carries collab fields.
- Affected files: none.
- Database impact: None.
- API impact: None.
- Frontend impact: None.

- Testing required: Existing personal-flow tests + manual smoke.
- Verification steps: Full personal workflow smoke test passes.
- Suggested commit message: `chore: personal-stack regression sign-off (R1)`

5. Success Criteria → Delivery Mapping

Success criterion	Delivered by
✓ Sprint model implemented	P1-T1
✓ Feature model implemented	P1-T2
✓ Project Backlog works via <code>sprintRef == null</code>	P1-T2 + P3-T3 + P6-T3
✓ Single Task collection supports collaboration	P1-T3 + P3-T5
✓ Backend APIs fully functional	Phase 3
✓ Collaboration store is API-backed	Phase 5
✓ UI uses live data	Phase 6
✓ Mock collaboration data removed	P5-T4
✓ Tests pass	Phase 7
✓ Personal-task functionality unaffected	Every phase regression + P8-T2

6. Risks & General Rollback

Risk	Mitigation	Rollback
<code>tasks.js</code> backward-compat regression	P2-T2 before P3-T5; personal query pinned to <code>workspaceRef: null</code> ; full regression each task	Revert P3-T5 (data unaffected)
Denormalized <code>workspaceRef</code> drift	Server-derived from Project (§9.3); never from client	n/a (prevention)
Migrations forward-only	Single release; DB backup before <code>--apply</code> ; run twice for idempotency	Manual <code>dropIndex / \$unset</code> + code revert
Store swap breaks call sites	Keep action return shapes compatible; Phase 6 updates consumers	Revert Phase 5/6 code

Risk	Mitigation	Rollback
Pages crash on empty/offline data	Empty/loading states in Phase 6 (mirror <code>loadProjects</code> offline fallback)	n/a
Mock <code>'m1'</code> leaks	P5-T4 grep assertion (<code>'m1'</code> count = 0)	n/a